

|                 |                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document number | D1819R0                                                                                                                                                                               |
| Date            | 2019-07-16                                                                                                                                                                            |
| Reply-to        | Vittorio Romeo < <a href="mailto:vittorio.romeo@outlook.com">vittorio.romeo@outlook.com</a><br>( <a href="mailto:vittorio.romeo@outlook.com">mailto:vittorio.romeo@outlook.com</a> )> |
| Audience        | Evolution Working Group Incubator (EWGI)                                                                                                                                              |
| Project         | ISO JTC1/SC22/WG21: Programming Language C++                                                                                                                                          |

# Interpolated literals

## Abstract

This paper proposes the addition of *interpolated literals* to the C++ language, a new form of string literals that can contain and retain arbitrary expressions. Interpolated literals provide a terse and readable syntax to generate strings embedding results of C++ expressions in a human-readable manner. The proposed feature is completely decoupled from any existing Standard Library facility, and provides a customization point to allow both Standard Library and user-defined types to visit the elements of an interpolated literal.

## Overview

One of the most common operations in any C++ code base is printing the value of a variable or the result of an expression alongside a human-readable message. Older techniques to achieve that goal, such as `<iostream>` and `printf`, have major drawbacks including **verbosity** and **lack of type-safety**. The

upcoming `<format>` header addresses some of those problems, but still requires the user to pass the expressions that need to be printed separately from the body of the literal<sup>1</sup>, and the formatting happens at run-time rather than compile-time.

This paper proposes a flexible *language* feature that does not intend to compete with `<format>` - instead, it intends to complete the set of C++ formatting tools available for users by providing a compile-time-friendly mechanism to process expressions embedded in string literal.

|                             | Execution    | i18n          | Expressions         |
|-----------------------------|--------------|---------------|---------------------|
| <code>&lt;format&gt;</code> | Run-time     | Supported     | Passed as arguments |
| Interpolated literals       | Compile-time | Not supported | Embedded in literal |

## In A Nutshell

Here’s how interpolated literals can be used to print a variable to stdout:

```
int port = 27015;
std::cout << f"Connecting on port {port}...\n";
//           ^~~~~~ ^~~~~ ^~~
//           literal piece | |
//                           expression piece
//                           |
//                           literal piece
```

Arbitrary expressions are supported as part of the interpolated literal:

```
int get_result() { return 42; }
std::cout << f"The result is {get_result()}\n";
```

# Syntax

- An interpolated literal always begins with the token `f`;
- Expressions can be embedded inside the literal by surrounding them with curly braces;
- Curly braces can be escaped by doubling them.

# Semantics

- An *interpolated literal expression* generates an **anonymous type** that satisfies the `InterpolatedLiteral` concept;
- The anonymous type is roughly equivalent to a *lambda expression* that captures all the embedded expressions by reference, and accepts a callable which is invoked with all the parts of the interpolated literal;
- The interpolated literal itself does nothing - it must be “consumed” by invoking it with a callable that performs a useful side effect (e.g. printing);
- An interpolated literal is purely syntactic sugar for the definition of an anonymous callable, similarly to lambda expressions.

# Examples

- Desugaring an interpolated literal to a lambda expression (#0):

```
f"Connecting on port {port}...\n"
// ...is roughly equivalent to...
[&](auto&& f) -> decltype(auto)
{
    return f("Connecting on port ", port, "...\\n");
}
```

- Note that the `decltype(auto)` trailing return type is essential to retain the value category of `f`'s return value.

- Desugaring an interpolated literal to a lambda expression (#1):

```
f"The result is {get_result()}\n";  
// ...is roughly equivalent to...  
[&](auto&& f) -> decltype(auto)  
{  
    return f("The result is ", get_result(), "\n");  
}
```

## Consuming an interpolated literal

Consumers of interpolated literals (e.g. `std::ostream`) can provide operator overloads accepting any object that satisfies `InterpolatedLiteral`. Note that the language feature is completely decoupled from any existing Standard Library facility. It is possible to use it with `<iostream>`, `<string>`, or any other user-defined type by simply defining appropriate overloads accepting `InterpolatedLiteral`.

### Examples

- Streaming an interpolated literal to `std::ostream`:

```
std::ostream& operator<<(std::ostream& os, const InterpolatedLiteral auto&  
    il)  
{  
    return il([&](const auto&... xs)  
    {  
        return (os << ... << xs);  
    });  
}
```

- Appending an interpolated literal to `std::string`:

```
std::string& operator+=(std::string& s, const InterpolatedLiteral auto&
    il)
{
    return il([&](const auto&... xs)
        {
            return (s += ... += xs);
        });
}
```

## Motivating examples

The main purpose of this feature is to allow users to conveniently print human-readable strings containing expressions.

## Debugging and logging

Interpolated literals are the easiest and tersest way to provide logs or debug print statements.

- Logging via macro with stream-like syntax:

```
void connect_to_host(net::ip ip, net::port port)
{
    LOG_TRACE << f"Connecting to {ip}:{port}...";
    // ...
}
```

- Printing debug information via `std::ostream`:

```
int main(int argc, char** argv)
{
    assert(argv == 3);
    std::cout << f"Input: {argv[1]}, Flags {argv[2]}\n";
}
```

- Logging via function:

```
if (const auto [k, v] item = get_item(); !item.is_valid())
{
    log::error(f"The item with key {k} and value {v} is not valid.");
    std::terminate();
}
```

- Passing an interpolated literal to another function to provide debug-only printing:

```
void debug_print([[maybe_unused]] const InterpolatedLiteral auto& il)
{
#ifdef NDEBUG
    std::cerr << il;
#endif
}

[[nodiscard]] long factorial(long n)
{
    debug_print(f"factorial({n})")

    if (n < 1) { return 1; }
    return n * factorial(n - 1);
}
```

## Automated coloring

If the feature is defined to automatically tag arguments as either *string literal pieces* or *embedded expressions*, it is possible to automatically color the embedded expressions when printing to a console. This can also be achieved by inspecting the type of the arguments passed to the higher-order function, at the cost of not being able to differentiate between string literal pieces and embedded expressions that evaluate to string literals. E.g.

```

struct pretty_cout { /* ... */ };

pretty_cout& operator+=(pretty_cout& os, const InterpolatedLiteral auto& il)
{
    il([&os](const auto&... xs)
    {
        ([&](const auto& x)
        {
            if constexpr (is_string_literal(x))
            {
                os << x;
            }
            else if constexpr (is_embedded_expression(x))
            {
                os << ansi_color::green << x << ansi_color::white;
            }
        })(xs, ...);
    });

    return os;
}

constexpr std::string_view name{"Bob"};
pretty_cout{} << f"Hello world! My name is {name}!\n";
    // "Bob" gets automatically colored in green in the console output.

```

## FAQ

- Why a callable object? Why not just capture things into a `std::tuple`?
  - Capturing this into a `std::tuple` introduces an undesirable dependency between the language feature and `<tuple>`. Furthermore, constructing a tuple from a function call is trivial (e.g. `il([](const auto&... xs){ return std::tuple{xs...}; })`), while expanding a tuple requires expensive compile-time machinery such as `std::apply`.

- Why a language feature? Can't we just wait for reflection?
  - This feature doesn't only require reflection, but also code synthesis. With reflection, we would need to be able to figure extract the variable names from the literals, then look them up, and then synthesize a lambda expression. This is not covered by the current Reflection TS and might take considerable time.
- Why not wait until `<format>` get `constexpr` formatting capabilities?
  - Even a fully-`constexpr` `<format>` library does not match the user-friendliness and intuitiveness of interpolated literals. Expressions cannot be embedded as part of the string literal unless we get extremely advanced reflection and code synthesis facilities (see above), and the formatting will be deeply tied to a particular library feature. The proposed language in this paper completely sidesteps these issues (and potential implementation complexities or compilation time explosion) by making interpolated literals a first-class feature. Note that this proposal does not compete with `<format>` - the `<format>` header is a great candidate for a consumer of interpolated literals.

## TODO

This proposal is just a draft to see EWGI likes the approach. If there is enough positive feedback, these are some points to think about:

- Tagging embedded expressions;

```
f"My name is {name}"  
// ...could expand to...  
[&](auto&& f) -> decltype(auto)  
{  
    return f(literal{"My name is "}, expression{name});  
}
```

- Dangling references concerns, maybe a syntax to capture rvalues by move;



```
f"My name is {name}"
    // Captures `name` by reference.

f[="My name is {name}"
    // Captures `name` by value, fully supports lambda syntax.
```

- Nested interpolated literals, passing string literals in curly braces;

```
f"Hello {\"world\"}"
    // Should probably work and evaluate to "Hello world".

f"Hello {f\"world\"(stringify)}"
    // Ditto.
```

- Formatting embedded expressions, for example floating point digits;

```
const double value = 12.34567;
const int width = 10;
const int precision = 4;

f"The result is {value:{width}.{precision}}"
    // Might provide information to evaluate result to "12.34".

// ...could expand to...

[&](auto&& f) -> decltype(auto)
{
    return f(literal{"The result is "},
             decimal_expression{value, width, precision});
}
```

- Standard Library support;
  - In the case of tagged elements, an `<interpolated_literal>` header might be provided by the Standard Library to expose the tag types.

- Use cases in the Standard Library (<iostream>, <string>, <format>);
  - Think about unconventional use cases.
- 

1. I.e. the expressions to format cannot be embedded directly in the literal. {} placeholders must instead be used, and the expressions have to be passed as arguments to the format call separately. See the [FAQ](#) section for more details.↵